Unidad 2
Encapsulamiento

Prof: Lic. Rosales Pablo (prosales@unpata.edu.ar)
Materia: Programación Orientada a Objetos
Año: 2026

Lo que acaban de hacer

- Antes: cuatro atributos `public`. Cualquier parte del programa escribía cualquier cosa y nadie protestaba.
- Después: la clase decide qué entra y qué no, y el resto del programa no puede hacer nada al respecto.

¿Qué imprimió el programa en la parte 1? ¿Quién tendría que haber impedido que ese perfil existiera?

Cuando terminaron, la parte 1 dejó de compilar. Ese error es la prueba de que funcionó: el programa perdió la posibilidad de escribir los atributos a mano.

¿Cuándo un perfil es válido?

Un perfil recién creado tiene el nombre completo vacío y el año de nacimiento en cero. ¿Es un estado válido?

- Cada clase hace una promesa sobre sus objetos, aunque no esté escrita en ningún lado.
- `PerfilUsuario`promete: un nombre de usuario que no está vacío y que no cambia, un email con arroba, un año que no es futuro.
- Lo que la promesa deja afuera también es una decisión de diseño, no un olvido.

Esa promesa es lo que hace confiable al objeto. Si se cumple siempre, el resto del programa puede usarlo sin desconfiar.

Encapsulamiento y ocultamiento de la información

Encapsulamiento

- Reunir en una sola entidad, la clase, los datos y las operaciones que trabajan sobre ellos.
- Esto ya lo hacíamos: métodos dentro de la clase, no funciones sueltas.

Ocultamiento de la información

- El estado interno queda fuera del alcance del resto del programa.
- Lo único visible es un conjunto de operaciones públicas, que funcionan como contrato.

Un cajero automático: nadie lo abre para sacar billetes de las cajas. Se pide una operación por la pantalla, y la máquina decide si se puede y es la única que toca el dinero.

Modificadores de acceso

Los dos de hoy

- `private`: solo desde adentro de la propia clase. Ahí van todos los atributos.
- `public`: desde cualquier clase. Ahí van las operaciones que la clase ofrece.

Los dos que vienen después

- `protected`: el mismo paquete y las subclases. Unidad 3, con herencia.
- `default`: lo que queda si no se escribe ningún modificador. Solo el mismo paquete.

La clase `Auto` de la clase 1 no declaraba nada, así que sus atributos eran `default`, no `public`. Alcanzaba con estar en el mismo paquete para escribirlos.

El constructor, y this

- Un método especial cuya única finalidad es dejar el objeto en un estado válido en el momento de crearlo.
- Se llama igual que la clase, no declara tipo de retorno, y se ejecuta solo al usar `new`.
- Si no escriben ninguno, Java crea uno sin parámetros. Apenas escriben el suyo, ese desaparece y `new PerfilUsuario()` deja de compilar.

`this.nombreUsuario = nombreUsuario;` a la izquierda el atributo, a la derecha el parámetro.

Sin `this`, las dos partes de esa línea se refieren al parámetro y la asignación no hace nada.

La interfaz pública: getters y setters

Getter

- Devuelve el valor de un atributo. Acceso de solo lectura.
- `getEmail()` deja ver el email, no cambiarlo.

Setter

- No es una asignación con más letras.
- Es donde la clase revisa el dato antes de aceptarlo.
- Si el valor rompe la promesa, no lo guarda.

Con los atributos en `private`, estas son las únicas puertas que quedan. Cada una hay que abrirla a propósito.

Cuando el dato no sirve

El setter devuelve false

- Rechaza el valor y deja el estado como estaba.
- No imprime: no sabe si lo llaman desde una consola, una ventana o un servidor.
- Quien lo llama decide cómo avisar.

El constructor corta

- No puede devolver `false`: ya está construyendo el objeto.
- Inventar un valor por defecto sería devolver un objeto que miente.
- `throw new IllegalArgumentException("...")`

El objeto se niega a nacer inválido. El programa corta en el punto exacto del error, en lugar de dejar circulando un perfil roto que va a fallar en otro lado.

Getters y setters no son un trámite

- El IDE los genera todos de un botón. Si cada atributo termina con setter público, el objeto quedó tan expuesto como antes, con el triple de líneas.

```
billetera.setSaldo(billetera.getSaldo() - 1000);
billetera.pagar(1000);
```

- En la primera decide el que llama, y puede olvidarse de revisar el saldo. En la segunda decide la billetera, que es la única que conoce sus reglas.

La pregunta no es qué atributos tiene la clase, sino qué operaciones ofrece. En `PerfilUsuario` no existe `setNombreUsuario`, y no es un olvido.

En resumen

- **Problema**: datos expuestos, y cualquiera los escribe.
- **Ocultar**: los atributos van `private`.
- **Crear**: constructor con validación, el objeto nace válido.
- **Mantener**: métodos públicos que revisan antes de aceptar.
- **Resultado**: una clase de la que se puede desconfiar bastante menos.

Ninguna de las cuatro sirve sola. `private` sin constructor deja objetos a medio hacer; el constructor sin `private` protege el nacimiento y nada más.

Un objeto que se hace cargo de sí mismo

Cada clase del proyecto integrador va a tener que responder lo mismo: qué promete, qué esconde y qué operaciones ofrece.